

ADAPTIVE CIVIC ROUTING INTELLIGENCE SYSTEM

How the Whole Application Works — Complete End-to-End Implementation Manual

HackMysuru 1.0 — Civic Routing Sub-Problem

System Notice: This document is an exhaustive, code-verified, engineering and operational explanation of the **Adaptive Civic Routing Intelligence System** as currently implemented. Every endpoint, database column, SQL query, React component, and Socket.IO event documented herein corresponds strictly to actual files in the repository.

TABLE OF CONTENTS

- 1. PART 1 — Project in One Simple Story
- 2. PART 2 — Big Connection Diagram
- 3. PART 3 — One Complaint: Complete Journey
- 4. PART 4 — Complaint Creation & Intake
- 5. PART 5 — AI-Assisted Issue Classification
- 6. PART 6 — Map, Location & Spatial Coordinate Handling
- 7. PART 7 — PostGIS Jurisdiction Spatial Verification
- 8. PART 8 — Authority & Department Routing Engine
- 9. PART 9 — Routing Decision & Version Provenance
- 10. PART 10 — Human Review & Operator Workspace
- 11. PART 11 — Case Status Lifecycle & State Machine
- 12. PART 12 — Service Level Agreements (SLA) & Escalation
- 13. PART 13 — Citizen Notification Engine
- 14. PART 14 — Real-Time WebSocket Gateway (Socket.IO)
- 15. PART 15 — Role-Based Access Control (Citizen vs. Operator vs. Admin)
- 16. PART 16 — Operational Analytics & Spatial Intelligence
- 17. PART 17 — Immutable Audit Trail & Security Auditing
- 18. PART 18 — Jurisdiction Versioning & Boundary Lifecycle
- 19. PART 19 — Complete Database Schema & Entity Relationships
- 20. PART 20 — Complete API Connection Map
- 21. PART 21 — Complete Source Code Connection Map
- 22. PART 22 — Manual "Follow One Complaint" Lab
- 23. PART 23 — Second Manual Lab: Human-in-the-Loop Review
- 24. PART 24 — What to Say to Judges
- 25. PART 25 — Common Judge Questions & Technical Answers
- 26. PART 26 — Production & Demonstration Troubleshooting
- 27. PART 27 — Final One-Page Architectural Cheat Sheet

PART 1 — PROJECT IN ONE SIMPLE STORY

Imagine a citizen standing on **Sayyaji Rao Road in Central Mysuru**. They see a large heap of unattended garbage blocking the sidewalk. They pull out their smartphone, open the civic web application, take a photo of the waste, enter their phone number (`9845012345`), and type:

"Garbage is overflowing from the corner bins near the market road."

As they type, the system's assistive **Google Gemini AI** reads the text and suggests the category `GARBAGE` with high confidence. The citizen taps the interactive Leaflet map, placing a precise GPS pin at Latitude `12.2958° N`, Longitude `76.6394° E`, and hits **"Submit Complaint"**.

The moment they click submit, the browser sends a multipart HTTP request to the Express backend. The backend stores the photo, verifies the Indian phone number, and assigns a monotonically increasing civic tracking code: `HM-CIV-2026-000656`.

Instantly, the database inserts this record into PostgreSQL, converting the raw coordinates into a true **PostGIS geometric point** (`SRID 4326`). The citizen immediately receives their confirmation code and a real-time notification.

Without requiring human triage, the system's **Deterministic Civic Routing Engine** activates. It does not guess, nor does it ask AI to pick an authority. Instead, it queries the database for the currently **ACTIVE jurisdiction version** (`MYS_2026_V1` or `MYS_2026_V2`) and executes a spatial query:

```
ST_Covers(jurisdiction.boundary, complaint.location)
```

The PostGIS spatial engine determines that the point falls squarely inside the **MCC Central Zone 1** polygon. This establishes that the responsible public authority is the **Mysuru City Corporation (MCC)**.

Next, the engine checks the relational database table `category_department_mappings`. It asks: *Under MCC, which department handles `GARBAGE`?*

The database answers: **Solid Waste Management (MCC_SWM)**.

A permanent, immutable record is written to `routing_decisions`, storing the exact jurisdiction version ID under which the decision was made. The complaint status advances from `SUBMITTED` to `ROUTED`. Simultaneously:

1. **The SLA Engine** initializes: Garbage has a strict 24-hour resolution target and an 18-hour warning threshold.
2. **The Notification Service** creates a persistent record: *"Your complaint has been routed to Mysuru City Corporation (Solid Waste Management)."*
3. **Socket.IO** broadcasts real-time events across the network, updating the citizen's screen, the operator's live feed, and public case tracking.
4. **The Immutable Audit Log** captures the transaction with full metadata and client IP, sealed by a PostgreSQL database trigger that prevents modification or deletion.

If the complaint had been filed outside all official jurisdiction boundaries, or if the category had been `OTHER`, the system would have safely marked it as `HUMAN_REVIEW`. The case would immediately appear in the **Operator Review Workspace**, where a human officer can inspect the map, read the notes, and assign the responsible authority with a single click.

Finally, the complaint flows into the **Operational Analytics Engine**. Without running heavy recalculations, executive charts update instantly, reflecting category distributions, authority workloads, spatial heatmaps, and SLA performance. When municipal boundaries shift in the future, version provenance guarantees that this complaint remains historically tied to MCC under the exact boundary version active on the day it was resolved.

PART 2 — BIG CONNECTION DIAGRAM



PART 3 — ONE COMPLAINT: COMPLETE JOURNEY

This section traces a single complaint from the moment a citizen types the description to its historical preservation in the audit trail.

- **Complaint Description:** "Garbage is being dumped near my street."
- **Selected Category:** GARBAGE
- **Attached Photo:** evidence_dump.jpg (Multipart binary upload)
- **Citizen Mobile:** +91 98450 12345 (Valid Indian phone format)
- **Coordinate Pin:** Latitude 12.2958, Longitude 76.6394 (Inside MCC Central Zone 1)

STEP 1 — CITIZEN INTAKE & AI ASSISTANCE

What the User Does

1. The user navigates to the **Citizen Portal** tab (<http://localhost:5173>).

- Types "Garbage is being dumped near my street" into the description textarea.
- Taps the "Classify with AI" button.

What the Frontend Does

- Component: `client/src/components/ComplaintForm.jsx`
- Calls `classifyComplaintText(description)` in `client/src/services/api.js`.
- Disables button and displays a loading spinner.

API Call

- Endpoint:** `POST /api/complaints/classify`
- Request:**

```
```json
{
 "description": "Garbage is being dumped near my street."
}
```
```

- Response:**

```
```json
{
 "success": true,
 "data": {
 "available": true,
 "category": "GARBAGE",
 "confidence": 0.95,
 "explanation": "Citizen describes uncollected domestic waste on street."
 }
}
```
```

Backend Handling

- Route:** `server/src/routes/complaintRoutes.js`
- Controller:** `server/src/controllers/complaintController.js` (`classifyText`)
- Service:** `server/src/services/aiService.js` (`classifyIssue`)
- Execution:** Calls the Google Gemini 1.5 Flash endpoint via HTTPS fetch using `process.env.GEMINI_API_KEY`. If the API key is not configured or network fails, returns `available: false` and prompts the user to select manually.

Database Impact

- None.** AI classification is completely stateless and read-only. No complaint rows are inserted yet.

What Happens Next

- `ComplaintForm.jsx` automatically selects the `GARBAGE` dropdown item and sets `category_source = 'AI_SUGGESTED'` and `category_confidence = 0.95`.

Manual Verification

- Type a sentence like "Streetlight is pitch dark" and click "Classify with AI". Check the Network tab in DevTools for `POST /api/complaints/classify` returning HTTP 200 with category `STREETLIGHT`.

What to Say to a Judge

"Our AI integration acts as a cognitive assistant for citizens, automatically mapping free-form colloquial descriptions to standardized municipal categories. Crucially, AI is assistive only—it does not assign jurisdiction, authority, or departments."

STEP 2 — LOCATION PIN & EVIDENCE SUBMISSION

What the User Does

- The user clicks on the interactive Leaflet map near Devaraja Market (Lat: `12.2958`, Lng: `76.6394`).
- Selects a JPEG photo from their device.
- Enters their 10-digit mobile number: `9845012345`.
- Clicks the "Submit Civic Complaint" button.

What the Frontend Does

- Component: `client/src/components/ComplaintForm.jsx`

- Validates that photo and phone are not empty.
- Prepares a `FormData` object containing:
 - * `description` : "Garbage is being dumped near my street."
 - * `category` : "GARBAGE"
 - * `category_source` : "AI_SUGGESTED"
 - * `category_confidence` : 0.95
 - * `latitude` : 12.2958
 - * `longitude` : 76.6394
 - * `citizen_contact` : "+919845012345"
 - * `photo` : Binary File object (`image/jpeg`)
- Calls `createComplaint(formData)` in `client/src/services/api.js`.

API Call

- Endpoint:** `POST /api/complaints`
- Headers:** `Content-Type: multipart/form-data`
- Response:**

```

{
  "success": true,
  "message": "Complaint registered successfully.",
  "data": {
    "complaint": {
      "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
      "complaint_code": "HM-CIV-2026-000656",
      "description": "Garbage is being dumped near my street.",
      "category": "GARBAGE",
      "category_source": "AI_SUGGESTED",
      "latitude": "12.2958000",
      "longitude": "76.6394000",
      "photo_url": "/uploads/complaints/photo-1726750000000.jpg",
      "status": "ROUTED",
      "citizen_contact": "+919845012345",
      "created_at": "2026-09-20T06:00:00.000Z"
    },
    "routingDecision": {
      "routing_status": "ROUTED",
      "authority": { "name": "Mysuru City Corporation (DEMO)", "code": "MCC_DEMO" },
      "department": { "name": "Solid Waste Management", "code": "MCC_SWM" }
    }
  }
}

```

Backend Handling

- Route:** `server/src/routes/complaintRoutes.js`
- Middleware:** `multer` middleware saves the photo to `server/uploads/complaints/` and validates that the MIME type is JPEG, PNG, or WebP.
- Controller:** `server/src/controllers/complaintController.js` (`createComplaint`)
- Service:** `server/src/services/complaintService.js` (`createComplaint`)

- Validates phone number via `validateAndNormalizeIndianPhone` into standard `+91XXXXXXXX` format.
- Runs spatial duplicate check within 100 meters over the last 7 days (`checkPotentialDuplicates`).
- Increments PostgreSQL sequence `complaint_code_seq` to generate `HM-CIV-2026-000656`.
- Inserts complaint record into `complaints` table.

5. Inserts initial intake entry into `complaint_status_history` (`SUBMITTED`).
6. Creates database notification `COMPLAINT_SUBMITTED` in `citizen_notifications` .
7. Emits `complaint:created` over Socket.IO.
8. Automatically calls `routingService.routeComplaint(complaint.id)` (Auto-Routing Pipeline).

Database Impact

- **Table:** `complaints`
 - * `id` : `f47ac10b-58cc-4372-a567-0e02b2c3d479`
 - * `complaint_code` : `'HM-CIV-2026-000656'`
 - * `location` : `ST_SetSRID(ST_MakePoint(76.6394, 12.2958), 4326)` (PostGIS Point Geometry)
 - * `status` : Initially `'SUBMITTED'` , immediately updated to `'ROUTED'`
- **Table:** `complaint_status_history`
 - * Added row: `previous_status: NULL` , `new_status: 'SUBMITTED'` , `changed_by: 'citizen_portal'` .
- **Table:** `citizen_notifications`
 - * Added row: `notification_type: 'COMPLAINT_SUBMITTED'` .

What Happens Next

- The auto-routing pipeline activates synchronously within the same request lifecycle.

Manual Verification

- Run in PostgreSQL:

```
```sql
SELECT complaint_code, category, status, ST_AsText(location)
FROM complaints WHERE complaint_code = 'HM-CIV-2026-000656';
```
```

- Expected output: `POINT(76.6394 12.2958)` and status `'ROUTED'` .

What to Say to a Judge

"When a complaint is submitted, we immediately bind the raw GPS coordinates to an indexed PostGIS geometry point in EPSG 4326. We enforce strict data hygiene with mandatory photo evidence and normalized phone validation before any routing computation begins."

STEP 3 — DETERMINISTIC POSTGIS JURISDICTION MATCHING

What the User Does

- *None (Automated backend transaction).*

What the Frontend Does

- Component: `client/src/components/ComplaintFeed.jsx`
- Receives real-time Socket.IO event `complaint:created` and prepends the new complaint card to the live intake feed.

API Call

- Internal service invocation: `routingService.routeComplaint(complaintId)` called from `complaintService.js` .

Backend Handling

- **Service:** `server/src/services/routingService.js` (`routeComplaint`)
- **Step 1: Check Active Version:**

Queries `jurisdiction_versions` where `status = 'ACTIVE'` (e.g., `MYS_2026_V1`).

- **Step 2: PostGIS Containment Query:**

Executes the spatial query:

```
```sql
SELECT
j.id AS jurisdiction_id,
j.name AS jurisdiction_name,
j.code AS jurisdiction_code,
a.id AS authority_id,
a.name AS authority_name
FROM complaints c
```

```

CROSS JOIN jurisdictions j
JOIN authorities a ON j.authority_id = a.id
WHERE c.id = 'f47ac10b-58cc-4372-a567-0e02b2c3d479'
AND jjurisdiction_version_id = 'c0000000-0000-0000-0000-000000000001'
AND ST_Covers(j.boundary, c.location)
ORDER BY j.created_at ASC
LIMIT 1;
...

```

- **PostGIS Result:** Match found! Covered by **MCC Central Zone 1** (`MCC_ZONE_1`), owned by **Mysuru City Corporation** (`a0000000-0000-0000-0000-000000000001`).

#### Database Impact

- **Read-Only Spatial Query:** PostGIS utilizes the GiST spatial index on `jurisdictions.boundary` to evaluate polygon containment in sub-millisecond time.

#### What Happens Next

- With the public authority established (`MCC`), the engine now resolves the internal departmental assignment.

#### Manual Verification

- Run in PostgreSQL:

```

```sql
SELECT j.name, a.name
FROM jurisdictions j
JOIN authorities a ON j.authority_id = a.id
WHERE ST_Covers(j.boundary, ST_SetSRID(ST_MakePoint(76.6394, 12.2958), 4326));
...

```

- Output: `MCC Central Zone 1 (DEMO) | Mysuru City Corporation (DEMO)`.

What to Say to a Judge

"We do not rely on fragile address strings or fuzzy reverse-geocoders. We execute an exact PostGIS `ST_Covers` spatial test against version-controlled municipal boundary polygons. If a coordinate is 1 millimeter inside MCC's boundary, it mathematically belongs to MCC."

STEP 4 — DEPARTMENTAL ASSIGNMENT & ROUTING DECISION PERSISTENCE

What the User Does

- *None (Automated backend transaction).*

Backend Handling

- **Service:** `server/src/services/routingService.js`
- **Department Lookup:**

Queries the database mapping table `category_department_mappings`:

```

```sql
SELECT d.id, d.name, d.code
FROM category_department_mappings cdm
JOIN departments d ON cdm.department_id = d.id
WHERE cdm.authority_id = 'a0000000-0000-0000-0000-000000000001'
AND cdm.category = 'GARBAGE';
...

```

Result: Department `d0000000-0000-0000-0000-000000000002` (**Solid Waste Management**).

- **Database Transaction ( `BEGIN` ... `COMMIT` ):**

1. Inserts record into `routing_decisions`.
2. Updates `complaints.status = 'ROUTED'` and `complaints.routed_at = CURRENT_TIMESTAMP`.
3. Inserts into `complaint_status_history` (`previous_status: 'SUBMITTED'`, `new_status: 'ROUTED'`).
4. Invokes `slaService.initializeComplaintSla`.
5. Inserts transaction-coupled audit log into `audit_logs` (`action: 'ROUTING_EXECUTED'`).

#### Database Impact

- **Table:** `routing_decisions`

```

* id : UUID
* complaint_id : f47ac10b-58cc-4372-a567-0e02b2c3d479
* jurisdiction_version_id : c0000000-0000-0000-0000-000000000001 (Preserves exact version provenance)
* jurisdiction_id : e0000000-0000-0000-0000-000000000001 (MCC Central Zone 1)
* authority_id : a0000000-0000-0000-0000-000000000001 (MCC)
* department_id : d0000000-0000-0000-0000-000000000002 (Solid Waste Management)
* routing_status : 'ROUTED'
* routing_method : 'GIS_RULE'
* reason : Full human-readable audit justification string.
* matched_at : CURRENT_TIMESTAMP

```

### What Happens Next

- The SLA tracking engine calculates warning and deadline timestamps based on the complaint category.

### Manual Verification

- Run in PostgreSQL:

```

```sql
SELECT rd.routing_status, rd.routing_method, a.name AS authority, d.name AS department, jv.version_code
FROM routing_decisions rd
JOIN authorities a ON rd.authority_id = a.id
JOIN departments d ON rd.department_id = d.id
JOIN jurisdiction_versions jv ON rd.jurisdiction_version_id = jv.id
WHERE rd.complaint_id = 'f47ac10b-58cc-4372-a567-0e02b2c3d479';
```

```

### What to Say to a Judge

*"Notice that the routing decision explicitly stores `jurisdiction_version_id`. Even if MCC's ward boundaries change next year, this complaint's historical routing decision remains permanently auditable and mathematically true to the day it occurred."*

## STEP 5 — SLA INITIALIZATION & RISK SCHEDULING

### What the User Does

- None (Automated backend transaction).

### Backend Handling

- **Service:** `server/src/services/slaService.js` ( `initializeComplaintSla` )
- Checks `complaint_sla_rules` for category `GARBAGE` :

```

* target_hours : 24
* warning_hours : 18

• Computes timestamps:

* sla_warning_at = routed_at + INTERVAL '18 hours'
* sla_target_at = routed_at + INTERVAL '24 hours'
* sla_status = 'WITHIN_SLA'

• Inserts row into append-only table complaint_sla_events .

```

### Database Impact

- **Table:** `complaints`

\* Updates `sla_warning_at` , `sla_target_at` , and sets `sla_status = 'WITHIN_SLA'` .

- **Table:** `complaint_sla_events`

```

* event_type : 'SLA_INITIALIZED'
* previous_sla_status : NULL
* new_sla_status : 'WITHIN_SLA'
* reason : 'SLA tracking initialized upon successful complaint routing'

```

### What Happens Next

- The notification engine dispatches a citizen alert, and Socket.IO broadcasts the update.

### Manual Verification

- Run in PostgreSQL:

```
```sql
SELECT complaint_code, routed_at, sla_warning_at, sla_target_at, sla_status
FROM complaints WHERE complaint_code = 'HM-CIV-2026-000656';
```
```

### What to Say to a Judge

"SLA tracking is not an afterthought calculated on page load; it is a transactional database guarantee. Timestamps are written directly to PostgreSQL columns upon routing, enabling automated query-driven breach detection."

## STEP 6 — CITIZEN NOTIFICATIONS & REAL-TIME WEBSOCKET BROADCAST

### What the User Does

- The citizen looks at their screen. A green notification badge appears on the bell icon in the top header.
- A toast alert appears: "Routing Completed — Your complaint has been routed to Mysuru City Corporation (Solid Waste Management)."

### What the Frontend Does

- Component: `client/src/components/Header.jsx` & `client/src/components/NotificationCenter.jsx`
- Listens to Socket.IO event `notification:created`.
- Increments unread badge count and adds the notification object to the dropdown tray.

### API Call

- When citizen clicks the bell icon:

```
GET /api/notifications
```

- When citizen clicks "Mark all as read":

```
POST /api/notifications/mark-all-read
```

### Backend Handling

- **Service:** `server/src/services/notificationService.js`

\* Inserts persistent record into `citizen_notifications` table:

\* `notification_type`: 'ROUTING\_COMPLETED'

\* `title`: 'Routing Completed'

\* `message`: 'Your complaint has been routed to Mysuru City Corporation (DEMO) (Solid Waste Management).'

- **Service:** `server/src/services/socketService.js`

\* Emits `io.emit('routing:completed', payload)`

\* Emits `io.emit('notification:created', notifPayload)`

\* Emits `io.emit('complaint:status_changed', statusPayload)`

### Database Impact

- **Table:** `citizen_notifications`

\* New record inserted with `is_read = FALSE` and idempotency key `ROUTING_COMPLETED:<complaint_id>`.

### Manual Verification

- Open the application in two separate browser windows (Window A = Citizen, Window B = Operator). Submit a complaint in Window A. Observe Window B's notification bell and feed update instantaneously without page refresh.

### What to Say to a Judge

"We maintain a clean decoupling between state persistence and event delivery. Notifications are first written to PostgreSQL with unique idempotency keys, and then dispatched through Socket.IO. If a user loses connection, their alerts remain safely stored in the database."

## STEP 7 — PUBLIC CASE TRACKING

### What the User Does

1. The user copies their tracking code: `HM-CIV-2026-000656`.
2. Clicks "Track Complaint" in the top navigation bar.

3. Enters `HM-CIV-2026-000656` into the lookup field and clicks Search.

### What the Frontend Does

- Component: `client/src/components/CaseTrackerModal.jsx`
- Calls `getComplaintByCode('HM-CIV-2026-000656')` in `client/src/services/api.js`.
- Calls `getComplaintRouting(complaintId)` to fetch routing details.
- Calls `getComplaintStatusHistory(complaintId)` to build the timeline.
- Calls `getComplaintSla(complaintId)` to display the SLA countdown.

### API Calls

- `GET /api/complaints/code/HM-CIV-2026-000656`
- `GET /api/complaints/:id/routing`
- `GET /api/complaints/:id/status-history`
- `GET /api/complaints/:id/sla`

### Database Impact

- Read-only queries joining `complaints`, `routing_decisions`, `authorities`, `departments`, and `complaint_status_history`.

### What the User Sees

- **Header:** Code, Category ( `GARBAGE` ), Status Badge ( `ROUTED` ).
- **SLA Card:** Target 24h, Remaining Hours, Green Progress Indicator ( `WITHIN_SLA` ).
- **Routing Card:** Authority ( `Mysuru City Corporation` ), Department ( `Solid Waste Management` ), Jurisdiction ( `MCC Central Zone 1` ), Version ( `MYS_2026_V1` ).
- **Status Timeline:**

\* `[11:30 AM]` — `SUBMITTED` (Initial citizen complaint registered)

\* `[11:30 AM]` — `ROUTED` (Deterministically routed by GIS Engine)

### What to Say to a Judge

"Citizens have full visibility without needing administrative credentials. The public case tracker exposes the complete lifecycle, SLA deadline, and jurisdictional provenance in an intuitive visual timeline."

## STEP 8 — OPERATOR WORKSPACE & STATUS ADVANCEMENT

### What the User Does (as Municipal Operator)

1. Clicks **"Demo Login"** in the top right, selects **"Login as Operator"**.
2. Navigates to the **"Deterministic Routing"** tab ( `RoutingView.jsx` ).
3. Enters `HM-CIV-2026-000656` to inspect the case.
4. Opens the **Case Tracker Modal** and updates the status:

Selects `IN_PROGRESS`, enters reason: "Field crew truck dispatched to location"\*, clicks **Update Status**.

Later, selects `RESOLVED`, enters reason: "Waste cleared and area sanitized"\*, clicks **Update Status**.

### What the Frontend Does

- Component: `client/src/components/CaseTrackerModal.jsx`
- Calls `updateComplaintStatus(complaintId, 'IN_PROGRESS', reason)` via `api.js`.
- Calls `updateComplaintStatus(complaintId, 'RESOLVED', reason)` via `api.js`.

### API Call

- **Endpoint:** `PATCH /api/complaints/:id/status`
- **Headers:** `Authorization: Bearer <operator_jwt_token>`
- **Request:**

```
```json
{
  "status": "IN_PROGRESS",
  "reason": "Field crew truck dispatched to location"
}
```

Backend Handling

- **Route:** `server/src/routes/complaintRoutes.js`
 - **Controller:** `server/src/controllers/caseStatusController.js` (`updateStatus`)
 - **Service:** `server/src/services/caseStatusService.js` (`transitionComplaintStatus`)
1. Validates status transition against the state machine (`ROUTED` -> `IN_PROGRESS` -> `RESOLVED`).

2. Updates `complaints.status`.
3. Appends new entry to `complaint_status_history` with `changed_by = 'operator@hackmysuru.gov.in'`.
4. Inserts `STATUS_CHANGED` notification into `citizen_notifications`.
5. Emits `complaint:status_changed` over Socket.IO.
6. Logs audit event `COMPLAINT_STATUS_CHANGED` in `audit_logs`.

Database Impact

- **Table:** `complaints` (`status = 'RESOLVED'`)
- **Table:** `complaint_status_history` (New audit row appended)
- **Table:** `audit_logs` (Security and operational log appended)

Manual Verification

- Run in PostgreSQL:

```
```sql
SELECT previous_status, new_status, changed_by, reason, created_at
FROM complaint_status_history
WHERE complaint_id = 'f47ac10b-58cc-4372-a567-0e02b2c3d479'
ORDER BY created_at ASC;
```
```

What to Say to a Judge

"Status transitions follow a strict, validated state machine. A case cannot jump arbitrarily from SUBMITTED to RESOLVED. Every state change is immutably logged with the actor's ID, role, timestamp, and justification."

STEP 9 — OPERATIONAL ANALYTICS AGGREGATION

What the User Does

- Operator clicks on the **"Analytics"** tab (`AnalyticsDashboard.jsx`).

What the Frontend Does

- Component: `client/src/components/AnalyticsDashboard.jsx`
- Calls all 10 analytics endpoints in parallel via `Promise.allSettled`.
- Renders metrics cards, bar charts, SLA doughnut charts, and the **Spatial Complaint Distribution** Leaflet map.

API Calls

- `GET /api/analytics/overview`
- `GET /api/analytics/categories`
- `GET /api/analytics/authorities`
- `GET /api/analytics/spatial`
- `GET /api/analytics/sla`

Backend Handling

- **Controller:** `server/src/controllers/analyticsController.js`
- **Service:** `server/src/services/analyticsService.js`
- Executes performant, read-only SQL aggregation queries across `complaints`, `routing_decisions`, `authorities`, and `departments`.
- `getSpatialComplaintDistribution` generates a GeoJSON Feature Collection where complaint `HM-CIV-2026-000656` is mapped at `[76.6394, 12.2958]` with properties containing authority, department, status, and SLA health.

What the User Sees

- Total complaints count incremented by 1.
- `GARBAGE` category count incremented by 1.
- MCC authority volume updated.
- On the Spatial Map, a marker appears at Devaraja Market; clicking it reveals `HM-CIV-2026-000656`, Solid Waste Management, Status: `RESOLVED`.

What to Say to a Judge

"Our analytics layer is completely non-blocking and read-only. It reads directly from indexed relational and spatial tables to produce real-time executive dashboards without degrading transactional intake throughput."

STEP 10 — IMMUTABLE AUDIT TRAIL PRESERVATION

What the User Does

- Admin logs in and opens the "Audit Logs" tab (AuditLogDashboard.jsx).

What the Frontend Does

- Component: client/src/components/AuditLogDashboard.jsx
- Calls getAuditLogs({ limit: 50 }) via api.js .
- Displays a filterable table of security and business events.

API Call

- GET /api/audit?limit=50

Backend Handling

- Controller: server/src/controllers/auditController.js
- Service: server/src/services/auditService.js (queryAuditLogs)
- Enforces server-side RBAC: Operators can view operational logs (COMPLAINT , ROUTING_DECISION , SLA); Admins can view all logs including AUTH_LOGIN and SECURITY .
- Automatically redacts sensitive parameters (passwords, JWT tokens).

Database Impact

- Table: audit_logs
- Rows corresponding to HM-CIV-2026-000656 :
 - action: 'COMPLAINT_CREATED' , entity_type: 'COMPLAINT'
 - action: 'ROUTING_EXECUTED' , entity_type: 'ROUTING_DECISION'
 - action: 'SLA_INITIALIZED' , entity_type: 'SLA'
 - action: 'COMPLAINT_STATUS_CHANGED' , entity_type: 'COMPLAINT'

- PostgreSQL Immutability Guarantee:

If any user or compromised process attempts:

```
```sql
DELETE FROM audit_logs WHERE entity_id = 'f47ac10b-58cc-4372-a567-0e02b2c3d479';
```
```

PostgreSQL throws an immediate runtime exception:

```
ERROR: audit_logs records are immutable and cannot be updated or deleted.
```

What to Say to a Judge

"We protect system integrity at the database engine level. Even a database administrator with full credentials cannot alter or delete rows in audit_Logs because PostgreSQL triggers actively reject any UPDATE or DELETE commands."

PART 4 — COMPLAINT CREATION & INTAKE

Architecture & Design

The intake module (ComplaintForm.jsx) provides a seamless citizen reporting interface backed by strict validation and spatial encoding.

Mandatory Field Requirements

To prevent spam, abuse, and orphaned reports, the system strictly enforces two mandatory fields:

- 1. Mandatory Photo Evidence:
 - * Must be uploaded as a binary file via multipart/form-data .
 - * Accepted formats: image/jpeg , image/png , image/webp .
 - * Size limit: Enforced at 5 Megabytes by Multer.
 - * Both MIME type and file extension are validated before writing to disk (server/uploads/complaints/).
- 1. Mandatory Indian Mobile Phone Number:
 - * Validated using regex: /^(?:\+91|91|0)?([6-9]\d{9})\$/ .
 - * Must start with digits 6, 7, 8, or 9 and contain exactly 10 digits.
 - * Normalized automatically into international E.164 format: +91XXXXXXXXXX .

Complaint Code Monotonicity

Complaint codes follow the pattern:

```
HM-CIV-2026-XXXXXX
```

Generated using the native PostgreSQL sequence `complaint_code_seq`:

```
SELECT nextval('complaint_code_seq') AS seq;
```

This guarantees unique, sequentially ordered, non-colliding human-readable tracking identifiers under high concurrency.

PostGIS Point Geometry Insertion

The backend converts the float latitude and longitude into an EPSG:4326 PostGIS geometry point:

```
INSERT INTO complaints (
  complaint_code, description, category, category_source,
  location, latitude, longitude, citizen_contact, photo_url, status
) VALUES (
  $1, $2, $3, $4,
  ST_SetSRID(ST_MakePoint($7, $8), 4326),
  $8, $7, $9, $6, 'SUBMITTED'
);
```

(Note: PostGIS `ST_MakePoint` takes Longitude ($\$7$, X) first, followed by Latitude ($\$8$, Y)).

Advisory Duplicate Warning System

Before insertion, `checkPotentialDuplicates` queries for complaints within **100 meters** of the same category filed within the last **7 days**:

```
SELECT complaint_code, created_at,
  ROUND(ST_Distance(location::geography, ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography)::numeric, 1) AS distance_meters
FROM complaints
WHERE category = $3
  AND created_at >= CURRENT_TIMESTAMP - INTERVAL '7 days'
  AND ST_DWithin(location::geography, ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography, 100);
```

This warning is **advisory only**. The complaint is never rejected; metadata simply notes `duplicateWarningIssued: true`.

PART 5 — AI-ASSISTED ISSUE CLASSIFICATION

Core Architectural Principle

AI is ASSISTIVE ONLY.

Google Gemini classifies only the civic issue type from our controlled category list.

AI NEVER decides jurisdiction, authority, department, or SLA.

Controlled Civic Categories

The system restricts issue classification strictly to eight controlled categories:

- `GARBAGE` — Uncollected domestic waste, street litter, overflowing bins.
- `ILLEGAL_DUMPING` — Commercial debris dumping, vacant lot waste piles.
- `POTHOLE` — Damaged asphalt, road craters, street depressions.
- `DRAINAGE` — Blocked storm gutters, overflowing sewage manholes.
- `STREETLIGHT` — Dark poles, non-functioning fixtures, flickering lamps.
- `C_AND_D_WASTE` — Construction and demolition masonry rubble on roadways.
- `WATER_LEAK` — Ruptured potable water mains, surface pipeline leaks.
- `OTHER` — Uncategorized issues requiring manual human triage.

Gemini API Integration Details

- File:** `server/src/services/aiService.js`
- Model:** `gemini-1.5-flash` (with fallback candidates `gemini-3.5-flash`, `gemini-flash-latest`).
- Protocol:** Direct HTTPS `fetch` using official Google API header `x-goog-api-key`.
- Configuration:** `temperature: 0.1` (deterministic), `response_mime_type: "application/json"`.
- Multimodal:** If an image is provided, reads the buffer and sends base64 image data alongside the text prompt.

Fallback Behavior (Zero-Crash Guarantee)

If `GEMINI_API_KEY` is not present in `.env`, or if the API returns an error:

- The service does **NOT** crash.
- The service does **NOT** invent fake categories.
- It returns `{ available: false, reason: "...", }`.

4. The frontend gracefully unlocks manual category selection with zero disruption to the citizen.

How to Test AI Classification Manually

1. Open Citizen Portal.
2. Enter: *"Pipe burst on roadside spilling clean drinking water."*
3. Click **"Classify with AI"**.
4. Observe dropdown auto-switch to `WATER_LEAK` with high confidence (~0.95).

PART 6 — MAP, LOCATION & SPATIAL COORDINATE HANDLING

Leaflet & OpenStreetMap Foundation

- **Component:** `client/src/components/Map.jsx`
- **Tiles:** Standard OpenStreetMap raster tiles (`https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`).
- **Tile Policy:** Completely free of proprietary keys (no Mapbox, no Google Maps API keys, no CartoDB API keys).

PostGIS Coordinate Order: Longitude First

A common pitfall in spatial programming is coordinate axis inversion:

- Human intuition and GPS devices say: **(Latitude, Longitude)** e.g., `(12.2958, 76.6394)` .
- Mathematical GIS systems (Cartesian X/Y) require: **(Longitude, Latitude)** e.g., `X = 76.6394, Y = 12.2958` .

PostGIS `ST_MakePoint(X, Y)` expects `ST_MakePoint(longitude, latitude)` .

Our codebase explicitly handles this:

```
ST_SetSRID(ST_MakePoint(longitude, latitude), 4326)
```

Setting SRID to `4326` assigns the standard **WGS 84** spatial reference system used globally by GPS.

Spatial GiST Indexing

Both `jurisdictions.boundary` and `complaints.location` are indexed using Generalized Search Trees (**GiST**):

```
CREATE INDEX idx_complaints_location ON complaints USING GIST (location);
CREATE INDEX idx_jurisdictions_boundary ON jurisdictions USING GIST (boundary);
```

This enables bounding-box intersection calculations in $O(\log N)$ time rather than full table scans.

PART 7 — POSTGIS JURISDICTION SPATIAL VERIFICATION

What is a Jurisdiction?

A jurisdiction represents a demarcated municipal polygon on earth owned by a public authority (e.g., MCC Central Zone 1).

The Jurisdiction Versioning Lifecycle

Boundaries change as cities grow. To manage changes without data corruption, the system implements a strict lifecycle in `jurisdiction_versions` :

- **DRAFT** : A new boundary version under development or undergoing geometry validation. Cannot route live complaints.
- **ACTIVE** : The single authoritative boundary version currently used for real-time routing.
- **RETIRED** : A previous active version, retained forever for historical audit and provenance.
- **ARCHIVED** : Deprecated version.

The Single Active Version Constraint

Enforced at the PostgreSQL engine level via a partial unique index:

```
CREATE UNIQUE INDEX uq_single_active_version
ON jurisdiction_versions (status)
WHERE status = 'ACTIVE';
```

It is physically impossible for the database to contain more than one active jurisdiction version simultaneously.

The Spatial Query (`ST_Covers`)

To determine which jurisdiction covers a complaint, `routingService.js` executes:

```
SELECT j.id, j.name, j.code, a.id AS authority_id, a.name AS authority_name
FROM complaints c
CROSS JOIN jurisdictions j
JOIN authorities a ON j.authority_id = a.id
WHERE c.id = $1
AND j.jurisdiction_version_id = $2
AND ST_Covers(j.boundary, c.location)
LIMIT 1;
```

Why `ST_Covers` instead of `ST_Contains` ?

In PostGIS, `ST_Contains` returns `false` if a point lies exactly on the polygon's exterior boundary line. `ST_Covers` returns `true` even if the complaint is directly on the boundary edge, eliminating boundary-line edge dropouts.

PART 8 — AUTHORITY & DEPARTMENT ROUTING ENGINE

Dynamic Decoupling (Why We Don't Hardcode "Garbage = MCC")

In legacy civic systems, developers write hardcoded logic:

```
// WRONG ARCHITECTURE:
if (category === 'GARBAGE') routeTo('MCC');
```

This fails in the real world:

- In Central Mysuru, `GARBAGE` is handled by **MCC Solid Waste Management**.
- In peripheral urban sectors, `GARBAGE` is handled by **MUDA Sector Maintenance**.
- If a new development layout is handed over from MUDA to MCC, hardcoded logic breaks.

Database-Driven Mapping Architecture

Our system decouples routing into two independent dimensions:

1. **Spatial Dimension:** Location determines **Authority** via PostGIS (`ST_Covers`).
2. **Category Dimension:** Category determines **Department** via `category_department_mappings` :

...

(Authority ID, Category) --> Department ID

...

```
[Complaint Location] — PostGIS ST_Covers —> [Authority: MCC]
                                     |
                                     v
[Complaint Category: GARBAGE] —————> [category_department_mappings]
                                     |
                                     v
                                   [Department: Solid Waste Management]
```

Deterministic Routing States

- `ROUTED` : Coordinate is covered by an active jurisdiction AND the category has a mapped department. Routing method = `GIS_RULE` .
- `HUMAN_REVIEW` :

* Condition A: Coordinate is outside all active jurisdiction boundaries.

* Condition B: Coordinate is covered, but category has no department mapping under that authority (e.g., category `OTHER`).

Routing method = `HUMAN_REVIEW` .

- `UNROUTABLE` : Marked by an operator when a complaint cannot be resolved or is invalid.

PART 9 — ROUTING DECISION & VERSION PROVENANCE

Schema of `routing_decisions`

Every routing computation persists a permanent decision row:

```
CREATE TABLE routing_decisions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  complaint_id UUID NOT NULL REFERENCES complaints(id) ON DELETE CASCADE,
  jurisdiction_version_id UUID NOT NULL REFERENCES jurisdiction_versions(id),
  jurisdiction_id UUID REFERENCES jurisdictions(id),
  authority_id UUID REFERENCES authorities(id),
  department_id UUID REFERENCES departments(id),
  routing_status VARCHAR(30) NOT NULL,
  routing_method VARCHAR(30) NOT NULL,
  reason TEXT NOT NULL,
  matched_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_complaint_routing UNIQUE (complaint_id)
);
```

Historical Immutability Example

Consider coordinate `(12.3300, 76.6000)`:

1. **Under Version 1 (`MYS_2026_V1`)**: The coordinate is inside MCC Central Zone.

* Complaint #101 is routed to **MCC**.

* `routing_decisions` records `jurisdiction_version_id = V1_UUID`.

1. **Boundary Revision**: Mysuru expands. Version 2 (`MYS_2026_V2`) is activated, assigning this sector to MUDA.
2. **Under Version 2 (`MYS_2026_V2`)**:

* Complaint #102 is filed at the exact same coordinate `(12.3300, 76.6000)`.

* Complaint #102 is routed to **MUDA**.

* `routing_decisions` records `jurisdiction_version_id = V2_UUID`.

The Guarantee: When reviewing Complaint #101 next year, querying its routing decision continues to show MCC under V1. The historical integrity of past municipal actions is completely preserved.

PART 10 — HUMAN REVIEW & OPERATOR WORKSPACE

When is Human Review Triggered?

Automatic routing halts safely and enters Human Review under two strict conditions:

1. **Spatial Boundary Mismatch**: The coordinates do not fall inside any polygon of the active boundary version.
2. **Category Ambiguity**: The category is `OTHER`, or no department mapping exists for `(authority_id, category)`.

Human Review Workflow

```
[Unmapped Complaint]
  ↓
[routingService] — Sets status = 'HUMAN_REVIEW'
  |
  |→ Creates complaint_reviews record (status: 'OPEN')
  |→ Emits Socket.IO 'routing:review_required' & 'review:created' to room 'privileged_operators'
  |→ Dispatches citizen notification 'HUMAN_REVIEW_REQUIRED'
```

Operator Review Actions

In `client/src/components/ReviewWorkspace.jsx`, authorized operators can perform three distinct actions:

1. **Start Review (`POST /api/reviews/:id/start`)**:

Claims the review. Status moves from `OPEN` to `IN_REVIEW`.

1. **Resolve Review (`POST /api/reviews/:id/resolve`)**:

Operator selects Authority, Department, and optional Jurisdiction override.

* Updates `complaint_reviews.review_status = 'RESOLVED'`.

* Updates `routing_decisions` with selected authority and department, setting `routing_method = 'HUMAN_REVIEW'`.

* Updates `complaints.status = 'ROUTED'`.

* Initializes SLA.

* Appends action to `complaint_review_actions` (`action_type: 'ROUTE_TO_AUTHORITY'`).

1. **Mark Unroutable (`POST /api/reviews/:id/reassign`)**:

Marks case as `UNROUTABLE` with justification. Status moves to `REJECTED`.

PART 11 — CASE STATUS LIFECYCLE & STATE MACHINE

The Formal State Machine

The system implements a controlled case status lifecycle in `server/src/services/caseStatusService.js`:



Allowed Transitions Table

| Current Status | Permitted Next Statuses | Actor Role Required |
|----------------|-----------------------------------|---------------------------|
| SUBMITTED | ROUTED , HUMAN_REVIEW , REJECTED | System / Operator / Admin |
| HUMAN_REVIEW | ROUTED , REJECTED | Operator / Admin |
| ROUTED | IN_PROGRESS , RESOLVED , REJECTED | Operator / Admin |
| IN_PROGRESS | RESOLVED , REJECTED | Operator / Admin |
| RESOLVED | CLOSED , IN_PROGRESS (Reopen) | Operator / Admin |
| CLOSED | Terminal (No further transitions) | None |
| REJECTED | Terminal (No further transitions) | None |

Append-Only Status Audit

Every transition inserts a row into `complaint_status_history`:

```
INSERT INTO complaint_status_history (
  complaint_id, previous_status, new_status, changed_by, reason, metadata
) VALUES ($1, $2, $3, $4, $5, $6);
```

PART 12 — SERVICE LEVEL AGREEMENTS (SLA) & ESCALATION

SLA Benchmark Configuration

SLA benchmarks are configured in `complaint_sla_rules` for all eight civic categories:

| Category | Target Hours (Breach) | Warning Hours (At Risk) | Scope / Urgency |
|-----------------|-----------------------|-------------------------|-------------------------------------|
| WATER_LEAK | 12 h | 8 h | Critical potable pipeline bursts |
| STREETLIGHT | 24 h | 16 h | Public darkness & electrical hazard |
| GARBAGE | 24 h | 18 h | Solid waste collection points |
| DRAINAGE | 48 h | 36 h | Storm drain blocks & sewage |
| ILLEGAL_DUMPING | 48 h | 36 h | Vacant layout dumping |
| POTHOLE | 72 h | 48 h | Asphalt failure & road craters |
| C_AND_D_WASTE | 72 h | 48 h | Construction debris clearance |
| OTHER | 96 h | 72 h | General civic inquiries |

SLA Statuses

- `WITHIN_SLA`: Elapsed time is less than warning hours threshold.
- `AT_RISK`: Elapsed time exceeds warning hours but is within target hours.
- `SLA_BREACHED`: Elapsed time exceeds target hours.

Escalation Engine (`checkSlaEscalations`)

Can be triggered periodically via cron or on-demand:

1. Queries complaints where `sla_status = 'WITHIN_SLA'` and `CURRENT_TIMESTAMP >= sla_warning_at`.
- * Updates `sla_status = 'AT_RISK'`.
 - * Inserts into `complaint_sla_events` (`event_type = 'SLA_WARNING'`).
 - * Emits `sla:warning` over Socket.IO.
1. Queries complaints where `sla_status != 'SLA_BREACHED'` and `CURRENT_TIMESTAMP >= sla_target_at`.
- * Updates `sla_status = 'SLA_BREACHED'`, sets `sla_breached_at = CURRENT_TIMESTAMP`.
 - * Inserts into `complaint_sla_events` (`event_type = 'SLA_BREACHED'`).
 - * Emits `sla:breached` over Socket.IO.
-

PART 13 — CITIZEN NOTIFICATION ENGINE

The 9 Controlled Notification Types

Persisted in `citizen_notifications` under strict check constraints:

1. `COMPLAINT_SUBMITTED` — Issued immediately upon valid intake.
2. `CATEGORY_UPDATED` — Issued if operator reclassifies the category.
3. `ROUTING_COMPLETED` — Issued upon successful automatic or manual routing.
4. `HUMAN_REVIEW_REQUIRED` — Issued if complaint requires officer intervention.
5. `STATUS_CHANGED` — Issued when status advances (e.g., `IN_PROGRESS`).
6. `SLA_WARNING` — Issued when 75% of resolution window has elapsed.
7. `SLA_BREACHED` — Issued when official resolution window has passed.
8. `CASE_RESOLVED` — Issued upon field resolution.
9. `CASE_CLOSED` — Issued upon administrative case closure.

Database Notification vs. Socket.IO Event

- **Database Notification** (`citizen_notifications`):

Permanent record with an unread boolean (`is_read = FALSE`). Guaranteed delivery across device reloads.

- **Socket.IO Event** (`notification:created`):

Transient real-time payload dispatched over WebSocket to pop active toast alerts and update unread count badges instantly.

PART 14 — REAL-TIME WEBSOCKET GATEWAY (SOCKET.IO)

WebSocket Architecture & Security

- **Server File:** `server/src/services/socketService.js`
- **Transport:** WebSocket with HTTP long-polling fallback.
- **Authentication Handshake:**

Inspects JWT from either `socket.handshake.auth.token` or HTTP cookie. If valid, binds user identity to the socket session.

Room Partitioning & Scoped Delivery

Clients are automatically segmented into rooms upon connection:

- `privileged_operators` : Joined only by authenticated `OPERATOR` and `ADMIN` users. Receives sensitive operational feeds.
- `role:CITIZEN` : Joined by citizens. Receives personal case updates.
- `role:ANONYMOUS` : Public unauthenticated clients.

Complete Inventory of Implemented Socket Events

| Event Name | Scope / Target Room | Purpose |
|--------------------------------|----------------------|--|
| system:connected | Connecting Socket | Acknowledges gateway handshake and reports user role |
| system:ping / pong | Connecting Socket | Heartbeat / latency probe |
| complaint:created | Broadcast (All) | Updates live intake feeds |
| complaint:status_changed | Broadcast (All) | Updates case timelines and tracking views |
| routing:completed | Broadcast (All) | Notifies of authority and department assignment |
| routing:review_required | privileged_operators | Alerts operators of an unmapped complaint |
| review:created | privileged_operators | Adds row to Operator Review Queue |
| review:updated | privileged_operators | Signals review claimed (IN_REVIEW) |
| review:resolved | privileged_operators | Removes row from Operator Review Queue |
| review:unroutable | privileged_operators | Signals case marked unroutable |
| notification:created | Broadcast (All) | Displays toast alert and updates bell badge |
| sla:warning | Broadcast (All) | Warns case is nearing SLA deadline |
| sla:breached | Broadcast (All) | Signals case has exceeded SLA deadline |
| jurisdiction:version_activated | Broadcast (All) | Reloads active polygons across all maps |

PART 15 — ROLE-BASED ACCESS CONTROL (CITIZEN VS OPERATOR VS ADMIN)

RBAC Matrix

| Capability | Citizen (Public) | Operator | Admin |
|---|------------------|----------|-------|
| Submit Complaint (Photo + Phone) | Yes | Yes | Yes |
| Classify Issue via AI | Yes | Yes | Yes |
| Track Complaint by Code | Yes | Yes | Yes |
| View Live Intake Feed | Yes | Yes | Yes |
| Access Deterministic Routing View | No | Yes | Yes |
| Access Human Review Workspace | No | Yes | Yes |
| Claim & Resolve Human Reviews | No | Yes | Yes |
| Update Case Status (IN_PROGRESS , RESOLVED) | No | Yes | Yes |
| View Operational Analytics | No | Yes | Yes |
| View Operational Audit Logs | No | Yes | Yes |
| View Security & Auth Audit Logs | No | No | Yes |
| Upload & Validate Boundaries | No | No | Yes |
| Activate Jurisdiction Versions | No | No | Yes |
| Provision User Accounts | No | No | Yes |

Authentication Implementation

- **Hashing:** bcryptjs with 10 salt rounds.
- **Tokens:** HMAC-SHA256 signed JSON Web Tokens (JWT) with 24-hour expiration.
- **Cookie Handling:** Secure httpOnly cookie with SameSite=Lax (or None with HTTPS).
- **Header Support:** Also accepts Authorization: Bearer <token> for API clients and automated tests.
- **1-Click Demo Login:** POST /api/auth/demo-login accepts { role: 'CITIZEN' | 'OPERATOR' | 'ADMIN' } to allow seamless evaluation by judges without password typing.

PART 16 — OPERATIONAL ANALYTICS & SPATIAL INTELLIGENCE

Read-Only Aggregation Layer

The analytics module (server/src/services/analyticsService.js) exposes ten aggregated endpoints:

1. `GET /api/analytics/overview` — Total intake, routed rate (%), open reviews, active SLA breaches.
2. `GET /api/analytics/trends` — Daily and weekly intake volume with status distribution.
3. `GET /api/analytics/categories` — Complaint volume, percentage share, and resolution rates per category.
4. `GET /api/analytics/authorities` — Workload division between MCC and MUDA.
5. `GET /api/analytics/departments` — Departmental queue depth and active cases.
6. `GET /api/analytics/routing` — Distribution of `GIS_RULE` vs `HUMAN_REVIEW` routing methods.
7. `GET /api/analytics/sla` — Breakdown of cases: `WITHIN_SLA`, `AT_RISK`, `SLA_BREACHED`.
8. `GET /api/analytics/reviews` — Open vs in-progress vs resolved review counts and average review time.
9. `GET /api/analytics/jurisdictions` — Case distribution across geographic jurisdiction zones.
10. `GET /api/analytics/spatial` — GeoJSON Feature Collection mapping all complaints as spatial points with rich metadata.

How a New Complaint Reaches Analytics

Analytics does not maintain a separate data warehouse. It executes fast aggregate SQL queries over the live transactional tables:

```
SELECT
  c.category,
  COUNT(c.id)::int AS total_complaints,
  COUNT(CASE WHEN c.status = 'RESOLVED' THEN 1 END)::int AS resolved_complaints
FROM complaints c
GROUP BY c.category
ORDER BY total_complaints DESC;
```

Because queries run against indexed foreign keys and categories, any newly submitted or resolved complaint reflects in analytics on the very next fetch.

PART 17 — IMMUTABLE AUDIT TRAIL & SECURITY AUDITING

Database Engine-Level Immutability

Traditional software logs are stored in text files or regular database tables where an attacker or malicious employee can execute `UPDATE` or `DELETE` to hide their tracks.

Our system enforces immutability at the PostgreSQL engine level in `012_immutable_audit_logs.sql`:

```
CREATE OR REPLACE FUNCTION prevent_audit_log_modification()
RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'audit_logs records are immutable and cannot be updated or deleted';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_logs_immutable
BEFORE UPDATE OR DELETE ON audit_logs
FOR EACH ROW
EXECUTE FUNCTION prevent_audit_log_modification();
```

Any `UPDATE` or `DELETE` statement executed against `audit_logs` triggers an immediate database exception. Only `INSERT` and `SELECT` are permitted.

Sensitive Data Redaction

Before any audit record is persisted, `sanitizeAuditMetadata` recursively scrubs sensitive fields:

- Passwords, tokens, cookies, secrets, and API keys are replaced with `[REDACTED]`.
- Bearer strings and JWT signatures (`eyJ...`) are replaced with `[REDACTED_TOKEN]`.

Transactional Coupling

In critical operations (such as boundary activation or complaint routing), the audit log insert is passed the active database `client`. If the business transaction fails and rolls back, the audit log rolls back with it, preventing phantom audit trails.

PART 18 — JURISDICTION VERSIONING & BOUNDARY LIFECYCLE

The 3-Day Innovation Scenario

DAY 1: Baseline Version 1 (`MYS_2026_V1`) is ACTIVE

- Polygon 1: Central Mysuru Municipal Zone (MCC).
- Citizen submits complaint at Location X (`12.3300, 76.6000`).
- PostGIS matches MCC. Routed to **MCC Solid Waste Management**.
- Decision is recorded with `jurisdiction_version_id = V1_UUID`.

DAY 2: Municipal Expansion — Administrator Creates Version 2 (`MYS_2026_V2`)

- Admin uses **Version Manager** to upload a new GeoJSON boundary package.
- V2 expands MUDA's Northwest Sector to annex Location X.

- V2 is saved as **DRAFT**.
- The Admin uses "**Preview Route**" to test Location X against V2:

* Result: Shows that under V2, Location X will route to **MUDA**.

* **Crucial Guarantee:** Live complaints submitted on Day 2 continue to route to **MCC** because V1 remains the sole **ACTIVE** version.

DAY 3: Zero-Downtime Safe Version Activation

- Admin clicks "**Activate Version**".
- Backend executes an atomic PostgreSQL transaction:

1. `UPDATE jurisdiction_versions SET status = 'RETIRED', retired_at = CURRENT_TIMESTAMP WHERE status = 'ACTIVE';`

2. `UPDATE jurisdiction_versions SET status = 'ACTIVE', activated_at = CURRENT_TIMESTAMP WHERE id = V2_UUID;`

3. Records audit transition in `audit_version_transitions`.

4. Emits `jurisdiction:version_activated` over Socket.IO.

- Now, a new complaint submitted at Location X routes to **MUDA**.
 - **Day 1 Complaint #101:** Still points to V1_UUID. Its audit record and authority assignment (MCC) remain 100% unaltered.
-

Entity-Relationship Diagram



| | | |
|-------------------------|----------------|-------------------|
| previous_stat | event_type | notification_type |
| new_status | new_sla_status | is_read, message |
| +-----+ +-----+ +-----+ | | |

Follow One Complaint Through the Database

Run this single master query in PostgreSQL to inspect every related record for a single complaint:

```
SELECT
  c.complaint_code,
  c.category,
  c.status AS complaint_status,
  c.sla_status,
  ST_AsText(c.location) AS coordinates,
  rd.routing_status,
  rd.routing_method,
  jv.version_code AS jurisdiction_version,
  j.name AS jurisdiction_zone,
  a.name AS assigned_authority,
  d.name AS assigned_department,
  (SELECT COUNT(*) FROM complaint_status_history h WHERE h.complaint_id = c.id) AS status_transitions,
  (SELECT COUNT(*) FROM citizen_notifications n WHERE n.complaint_id = c.id) AS notifications_count,
  (SELECT COUNT(*) FROM audit_logs al WHERE al.entity_id = c.id::text OR al.metadata->>'complaintCode' = c.complaint_code) AS audit_records
FROM complaints c
LEFT JOIN routing_decisions rd ON rd.complaint_id = c.id
LEFT JOIN jurisdiction_versions jv ON rd.jurisdiction_version_id = jv.id
LEFT JOIN jurisdictions j ON rd.jurisdiction_id = j.id
LEFT JOIN authorities a ON rd.authority_id = a.id
LEFT JOIN departments d ON rd.department_id = d.id
WHERE c.complaint_code = 'HM-CIV-2026-000656';
```

PART 20 — COMPLETE API CONNECTION MAP

| Method | Endpoint | Authorization | Controller / Service | Primary Tables | Frontend Consumer |
|--------|--|----------------|---|--|-------------------------|
| POST | /api/complaints | Public | complaintController.createComplaint | complaints ,
routing_decisions ,
citizen_notifications | ComplaintForm.jsx |
| POST | /api/complaints/classify | Public | complaintController.classifyText | None (Gemini 1.5 API) | ComplaintForm.jsx |
| GET | /api/complaints | Public | complaintController.getComplaints | complaints | ComplaintFeed.jsx |
| GET | /api/complaints/:id | Public | complaintController.getComplaintById | complaints | CaseTrackerModal.js |
| GET | /api/complaints/code/:code | Public | complaintController.getComplaintByCode | complaints | CaseTrackerModal.js |
| GET | /api/complaints/:id/routing | Public | routingController.getComplaintRouting | routing_decisions ,
authorities , departments | RoutingDecisionModal.js |
| POST | /api/complaints/:id/route | Operator/Admin | routingController.routeComplaint | routing_decisions ,
complaints | RoutingView.jsx |
| PATCH | /api/complaints/:id/status | Operator/Admin | caseStatusController.updateStatus | complaints ,
complaint_status_history | CaseTrackerModal.js |
| GET | /api/complaints/:id/status-history | Public | caseStatusController.getStatusHistory | complaint_status_history | StatusTimeline.jsx |
| GET | /api/complaints/:id/sla | Public | slaController.getComplaintSla | complaints ,
complaint_sla_events | SlaStatusCard.jsx |
| GET | /api/routing/summary | Operator/Admin | routingController.getRoutingSummary | routing_decisions | RoutingView.jsx |
| GET | /api/routing/feed | Operator/Admin | routingController.getRoutingFeed | routing_decisions ,
complaints | RoutingView.jsx |
| GET | /api/jurisdictions/active | Public | jurisdictionController.getActive | jurisdictions ,
jurisdiction_versions | Map.jsx |
| GET | /api/jurisdictions/versions | Operator/Admin | jurisdictionController.getVersions | jurisdiction_versions | VersionManager.jsx |
| POST | /api/jurisdictions/versions | Admin Only | jurisdictionController.createVersion | jurisdiction_versions ,
jurisdictions | VersionManager.jsx |
| POST | /api/jurisdictions/versions/:id/activate | Admin Only | jurisdictionController.activateVersion | jurisdiction_versions ,
audit_version_transitions | VersionManager.jsx |
| POST | /api/jurisdictions/preview-route | Operator/Admin | jurisdictionController.previewRoute | jurisdictions ,
category_dept_mappings | VersionManager.jsx |
| GET | /api/reviews/workspace | Operator/Admin | reviewController.getWorkspace | complaint_reviews ,
complaints | ReviewWorkspace.jsx |
| POST | /api/reviews/:id/start | Operator/Admin | reviewController.startReview | complaint_reviews ,
complaint_review_actions | ReviewWorkspace.jsx |
| POST | /api/reviews/:id/resolve | Operator/Admin | reviewController.resolveReview | complaint_reviews ,
routing_decisions | ReviewWorkspace.jsx |
| GET | /api/notifications | Public | notificationController.getNotifications | citizen_notifications | NotificationCenter.js |
| POST | /api/notifications/mark-all-read | Public | notificationController.markAllRead | citizen_notifications | NotificationCenter.js |
| GET | /api/analytics/overview | Operator/Admin | analyticsController.getOverview | complaints ,
routing_decisions | AnalyticsDashboard.js |
| GET | /api/analytics/spatial | Operator/Admin | analyticsController.getSpatial | complaints ,
routing_decisions | AnalyticsDashboard.js |
| GET | /api/audit | Operator/Admin | auditController.getAuditLogs | audit_logs | AuditLogDashboard.js |
| POST | /api/auth/login | Public | authController.login | users , audit_logs | AuthModal.jsx |
| POST | /api/auth/demo-login | Public | authController.demoLogin | users , audit_logs | AuthModal.jsx |
| GET | /api/auth/me | Authenticated | authController.getMe | users | AuthContext.jsx |

PART 21 — COMPLETE SOURCE CODE CONNECTION MAP

| Frontend Component | Action / Event | API Call | Backend Controller | Service Handler |
|------------------------|--------------------------|---|--|---------------------------------------|
| ComplaintForm.jsx | Click "Classify with AI" | POST /api/complaints/classify | complaintController.classifyText | aiService.classifyIssue |
| ComplaintForm.jsx | Click "Submit Complaint" | POST /api/complaints | complaintController.createComplaint | complaintService.createComplaint |
| RoutingView.jsx | Search Code | GET /api/complaints/:id/routing | routingController.getComplaintRouting | routingService.getRoutingDecisionByCo |
| ReviewWorkspace.jsx | Click "Start Review" | POST /api/reviews/:id/start | reviewController.startReview | reviewService.startReview |
| ReviewWorkspace.jsx | Click "Assign & Resolve" | POST /api/reviews/:id/resolve | reviewController.resolveReview | reviewService.resolveReview |
| CaseTrackerModal.jsx | Click "Update Status" | PATCH /api/complaints/:id/status | caseStatusController.updateStatus | caseStatusService.transitionComplaint |
| VersionManager.jsx | Click "Activate Version" | POST /api/jurisdictions/versions/:id/activate | jurisdictionController.activateVersion | versionService.activateVersion |
| NotificationCenter.jsx | Click Notification Item | PATCH /api/notifications/:id/read | notificationController.markRead | notificationService.markAsRead |
| AnalyticsDashboard.jsx | Mount Component | GET /api/analytics/spatial | analyticsController.getSpatial | analyticsService.getSpatialComplaintD |
| AuditLogDashboard.jsx | Filter Events | GET /api/audit | auditController.getAuditLogs | auditService.queryAuditLogs |

PART 22 — MANUAL "FOLLOW ONE COMPLAINT" LAB

Follow this exact physical trial to verify the entire system from your browser and terminal.

Phase 1: Verify Stack Readiness

1. Open PowerShell:
`powershell`
Check backend health
Invoke-RestMethod http://localhost:4000/api/health
``
- Verify that `data.database.connected` is `true` and `postgisInstalled` is `true`.
1. Open browser at `http://localhost:5173`.

Phase 2: File the Complaint

1. On the **Citizen Portal** (`ComplaintForm.jsx`):
- * **Description:** Type `"Large heap of rotten organic garbage dumped near roadside."`
- * **Category:** Click **"Classify with AI"**. Verify it selects `GARBAGE`.
- * **Phone:** Enter `9845012345`.
- * **Photo:** Attach any valid JPEG/PNG image.
- * **Location:** Click the map near Central Mysuru (Lat: `12.2958` , Lng: `76.6394`).

* Click **"Submit Civic Complaint"**.

1. Immediate UI Observation:

* A confirmation screen appears displaying:

Complaint Registered: HM-CIV-2026-XXXXXX

* Copy this complaint code!

Phase 3: Inspect Database Records

Open your PostgreSQL terminal (`psql -U postgres -d adaptive_civic_routing`) and run:

```
SELECT complaint_code, category, status, citizen_contact
FROM complaints
ORDER BY created_at DESC LIMIT 1;
```

Observe status is already `ROUTED` (auto-routing completed).

```
SELECT rd.routing_status, rd.routing_method, a.name AS authority, d.name AS department
FROM routing_decisions rd
JOIN authorities a ON rd.authority_id = a.id
JOIN departments d ON rd.department_id = d.id
ORDER BY rd.created_at DESC LIMIT 1;
```

Observe authority is `Mysuru City Corporation (DEMO)` and department is `Solid Waste Management`.

Phase 4: Inspect Live Operator Routing View

1. Log in as Operator (Click **Demo Login** -> **Login as Operator**).
2. Click the **"Deterministic Routing"** tab.
3. Paste your complaint code into the search box.
4. **UI Observation:** Shows full green card:

* Status: `ROUTED`

* Authority: `Mysuru City Corporation (DEMO)`

* Department: `Solid Waste Management`

* Method: `GIS_RULE`

* Version: `MYS_2026_V1`

Phase 5: Advance Status to Resolved

1. Click **"Track Complaint"** in header, paste the code.
2. Under Status Actions:

* Change to `IN_PROGRESS` -> Reason: `"Sanitation vehicle assigned"`.

* Change to `RESOLVED` -> Reason: `"Garbage cleared"`.

1. Check your notification bell: Notice that 3 real-time alerts were generated.

Phase 6: Verify Analytics & Audit

1. Click the **"Analytics"** tab.
2. Scroll to **Spatial Complaint Distribution**. Find your marker at `(12.2958, 76.6394)`. Click it to view the resolved details.
3. Click the **"Audit Logs"** tab.
4. Filter by Action: `COMPLAINT_CREATED` and `ROUTING_EXECUTED`. Observe the immutable audit trail entries containing your complaint code.

PART 23 — SECOND MANUAL LAB: HUMAN-IN-THE-LOOP REVIEW

This exercise demonstrates what happens when a complaint **cannot** be safely resolved automatically.

Phase 1: Submit an Out-of-Boundary Complaint

1. Open Citizen Portal.
2. Description: `"Construction debris dumped on highway past airport."`
3. Category: Select `C_AND_D_WASTE`.
4. Phone: Enter `9822011223`.
5. Photo: Attach image.
6. **Location:** Click far south outside the city boundaries (e.g., Lat: `12.1800`, Lng: `76.6000`).
7. Submit the complaint and record the code (e.g., `HM-CIV-2026-000657`).

Phase 2: Verify Automatic Flagging

1. Open PostgreSQL:

```
```sql
SELECT status FROM complaints WHERE complaint_code = 'HM-CIV-2026-000657';
```
```

Output: `HUMAN_REVIEW`.

1. Check routing decision reason:

```
```sql
SELECT reason FROM routing_decisions WHERE routing_status = 'HUMAN_REVIEW' ORDER BY created_at DESC LIMIT 1;
```
```

Reason: `"Complaint coordinates ... are outside all configured jurisdiction boundaries in active version MYS_2026_V1. Flagged for human review."`

Phase 3: Operator Resolution

- 1. Log in as Operator.
- 2. Navigate to the "Human Review" tab (`ReviewWorkspace.jsx`).
- 3. Notice your complaint is listed with status `OPEN`.
- 4. Click "Start Review". Status changes to `IN_REVIEW`.
- 5. In the resolution panel:

* Authority: Select `Mysuru Urban Development Authority (DEMO)`.

* Department: Select `Sector Maintenance`.

Notes: Type "Jurisdiction boundary extension authorized for peripheral bypass road."

* Click "Assign & Resolve".

Phase 4: Verify Resolution

- The case disappears from the Review Workspace.
- Status in `complaints` advances to `ROUTED`.
- Notification sent to citizen: "Routing Completed — Your complaint has been routed to Mysuru Urban Development Authority (Sector Maintenance)."
- Check `complaint_review_actions`: An append-only audit entry logs the operator's manual override.

PART 24 — WHAT TO SAY TO JUDGES

30-Second Elevator Pitch

"Most civic grievance systems fail because complaints sit in generic queues or rely on unverified text matching. We built the Adaptive Civic Routing Intelligence System for HackMysuru: an automated platform that combines assistive Gemini AI classification with deterministic PostGIS spatial polygon containment. The system mathematically proves which public authority owns an issue in sub-milliseconds, tracks strict SLAs, maintains immutable audit trails, and guarantees zero historical corruption when municipal boundaries change."

2-Minute Standard Demonstration

**Judges, let me show you what happens when a citizen files a complaint.

First, our system solves the data hygiene challenge: photos and validated Indian phone numbers are mandatory. As the citizen types, Google Gemini assists by standardizing the issue into controlled civic categories. But notice our core architectural distinction: AI never decides jurisdiction.

The moment the complaint is placed on the map, our PostGIS engine executes an exact `ST_Covers` spatial test against our active jurisdiction boundary polygons. If the coordinate is in MCC Central Zone, it goes to MCC. Then, our database routing matrix maps the category to the exact internal department—such as Solid Waste Management.

If an issue lies on an unmapped boundary or has an ambiguous category, the system never guesses—it safely routes to our Operator Human Review Workspace. Everything is real-time via Socket.IO, governed by role-based access control, benchmarked by category-specific SLAs, and sealed by database-level immutable audit triggers."

5-Minute Technical Deep Dive

**Judges, I want to direct your attention to the technical architecture of this platform:

- 1. **Spatial Determinism:** We do not use bounding boxes or radius circles. We use Open Geospatial Consortium (OGC) MultiPolygons with GiST spatial indexing in PostGIS.
- 2. **Boundary Versioning & Immutability:** Cities are dynamic. When a city expands, old complaints often get corrupted if boundaries are simply overwritten. We created a formal versioning lifecycle: Draft, Active, Retired. A PostgreSQL partial unique index guarantees that only one version can be active at a time. When a routing decision is recorded, it permanently references the active version ID. Even if boundaries shift tomorrow, past decisions remain mathematically provable.
- 3. **Non-Blocking Architecture:** Our spatial routing and SLA calculations happen synchronously within an ACID transaction, while heavy notifications and WebSocket events dispatch asynchronously.

4. **Database-Level Immutability:** Many teams build audit logs in software. We built ours into PostgreSQL itself. We attached a trigger that actively blocks any UPDATE or DELETE operations on `audit_logs`. Even if an attacker compromises the backend credentials, past records cannot be expunged."

10-Minute Full Demonstration Script

- **Minute 0–2:** Introduce the civic challenge in Mysuru (overlapping MCC vs MUDA jurisdictions, jurisdictional ping-pong, citizen frustration).
- **Minute 2–4:** Live Citizen Intake demonstration. Type description, trigger AI classification, drop pin on Sayyaji Rao Road, submit with photo. Show instant code generation (`HM-CIV-2026-XXXXXX`).
- **Minute 4–6:** Show the Deterministic Routing View. Explain `ST_Covers`, the active version provenance, and category-to-department matrix.
- **Minute 6–7:** Trigger Human Review by dropping a pin outside city boundaries. Open the Operator Review Workspace in a second tab. Claim and resolve the case live.
- **Minute 7–8:** Show Public Case Tracking. Point out the real-time SLA countdown timer and status timeline.
- **Minute 8–9:** Open Analytics and Audit Logs. Demonstrate the 200+ complaint spatial GeoJSON distribution map and show the database immutability trigger preventing audit log deletion.
- **Minute 9–10:** Conclusion, Q&A, and summary of civic impact.

PART 25 — COMMON JUDGE QUESTIONS & TECHNICAL ANSWERS

1. How does routing happen?

Routing is a two-step deterministic pipeline: first, PostGIS executes `ST_Covers(jurisdiction.boundary, complaint.location)` against the active boundary version to identify the Authority. Second, the database queries `category_department_mappings` with `(authority_id, category)` to identify the Department.

2. Why use PostGIS instead of simple latitude/longitude bounding boxes?

Real-world municipal wards are irregular, non-convex polygons defined by winding rivers, railways, and arterial roads. Bounding boxes or radius distances cause catastrophic false positives in border zones. PostGIS provides exact point-in-polygon computational geometry.

3. Why not use normal geocoding APIs like Google Maps Reverse Geocoding?

Reverse geocoding returns postal addresses (e.g., "Sayyaji Rao Road, Mysuru"). It does not know the internal administrative boundaries separating MCC from MUDA, nor does it know which municipal engineer oversees which sector.

4. How does the system know which authority owns a location?

Every spatial polygon in `jurisdictions` has a foreign key to `authorities(id)`. When `ST_Covers` matches the polygon, the owning authority is resolved directly from the relational join.

5. How does department routing happen?

Through the `category_department_mappings` table. It pairs each authority with our eight controlled categories to return the responsible department foreign key.

6. What happens if a coordinate falls outside all jurisdictions?

The system safely marks the routing decision as `HUMAN_REVIEW` with method `HUMAN_REVIEW`. It creates an open case in `complaint_reviews` and dispatches it to the Operator Review Workspace.

7. What happens if the Gemini AI service fails or is offline?

The application never crashes. The AI service returns `{ available: false }`, and the citizen reporting form immediately unlocks manual category selection.

8. Does AI make the routing decision?

No, absolutely not. AI is strictly assistive and only suggests the category. Jurisdiction, authority, and department routing are 100% deterministic and calculated by PostGIS and relational database rules.

9. How do you prevent old complaints from changing when boundaries change?

Every row in `routing_decisions` stores `jurisdiction_version_id`. Historical complaints point permanently to the version ID under which they were routed, ensuring historical immutability.

10. What is the purpose of jurisdiction versioning?

It allows administrators to draft, validate, and preview new municipal boundaries without disrupting live routing, and then activate changes with zero downtime in a single atomic transaction.

11. How does human review work?

Ambiguous complaints are queued in `complaint_reviews`. Authorized operators claim the case, inspect the map and photo, select the appropriate authority and department, and click resolve.

12. How does the operator interact with the complaint?

Through the Operator Review Workspace and Case Tracker Modal, where they can claim reviews, reassign departments, and transition case statuses (`IN_PROGRESS`, `RESOLVED`).

13. How does SLA tracking work?

Upon routing, `slaService` calculates `sla_warning_at` and `sla_target_at` using category-specific hours stored in `complaint_sla_rules`, persisting timestamps directly onto the complaint row.

14. How do notifications work?

Notifications are inserted into `citizen_notifications` and simultaneously broadcast via Socket.IO to display toast alerts and update unread count badges.

15. How does real-time updating work?

Using a centralized Socket.IO server (`socketService.js`) integrated with Node/Express that broadcasts events across partitioned rooms (`privileged_operators` , `role:CITIZEN`).

16. How does Analytics get its data?

Analytics queries live database tables using optimized aggregation queries (`GROUP BY` , `COUNT` , `JOIN`) and GeoJSON serialization without needing an external ETL pipeline.

17. How is the system auditable?

Every significant event writes to `audit_logs` . A PostgreSQL database trigger forbids any `UPDATE` or `DELETE` operations, making the audit trail completely tamper-proof.

18. How is citizen phone data protected?

Phone numbers are normalized, validated, and hidden from unauthenticated public feeds. Furthermore, audit metadata sanitation routines automatically redact sensitive tokens and credentials.

19. What does RBAC do?

Role-Based Access Control restricts administrative tabs (Routing Engine, Review Workspace, Boundary Management, Analytics, Audit) strictly to authenticated `OPERATOR` and `ADMIN` roles using JWT validation.

20. What makes this different from a simple complaint form?

A simple complaint form puts unstructured text into an email inbox or flat table. Our platform is an end-to-end civic intelligence engine: it automates boundary matching with PostGIS, guarantees data hygiene, manages boundary lifecycles, enforces SLAs, and provides tamper-proof auditability.

PART 26 — TROUBLESHOOTING GUIDE

| Symptom | Likely Cause | What to Check | How to Fix |
|---|--|--|--|
| "Cannot connect to database" | PostgreSQL service stopped or wrong credentials | Check <code>server/.env</code> for <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_PASSWORD</code> . | Run <code>net start postgresql-x64-16</code> or restart PostgreSQL in Windows Services. |
| "PostGIS extension missing" | PostGIS not enabled in database | Run <code>SELECT PostGIS_Version();</code> in psql. | Run <code>CREATE EXTENSION postgis;</code> in database. |
| "AI classification returns unavailable" | Missing or invalid <code>GEMINI_API_KEY</code> in <code>server/.env</code> | Check <code>server/.env</code> for <code>GEMINI_API_KEY</code> . | Add a valid Google Gemini API key or let citizens use manual selection. |
| "Map tiles fail to load" | Internet connection issue or blocked OpenStreetMap CDN | Check browser DevTools Network tab for tile 404/blocked requests. | Ensure workstation has active internet access to <code>tile.openstreetmap.org</code> . |
| "Photo upload fails with HTTP 400" | File size exceeds 5MB or invalid MIME type | Check uploaded file size and extension (must be JPEG, PNG, or WebP). | Upload an image under 5MB with valid image format. |
| "Phone validation error" | Non-Indian number or invalid format | Ensure number starts with 6, 7, 8, or 9 and has exactly 10 digits. | Enter standard Indian mobile format, e.g., <code>9845012345</code> . |
| "Complaint shows AWAITING ROUTING" | Routing engine did not auto-fire | Check if <code>autoRoute</code> flag was false or unhandled exception occurred. | Click "Re-Route Complaint" in <code>RoutingView.jsx</code> or check backend logs. |
| "Human Review queue empty" | No complaints flagged for review | Check <code>SELECT count(*) FROM complaint_reviews WHERE review_status = 'OPEN';</code> | Submit an out-of-boundary complaint to test review queuing. |
| "Socket.IO shows Disconnected" | Port 4000 blocked or CORS origin mismatch | Check browser console for WebSocket connection errors. | Verify backend is running on <code>http://localhost:4000</code> and <code>CLIENT_URL</code> matches frontend port. |
| "401 Unauthorized on Review tab" | Not logged in as Operator or Admin | Check auth state in top header. | Click Demo Login -> Select "Login as Operator". |
| "Cannot activate version: Overlap detected" | Uploaded polygons overlap spatially | PostGIS detected overlapping geometries between authorities. | Run geometry validation and repair polygon boundaries. |
| "Audit log deletion error" | Expected behavior! Database trigger blocked DELETE | Verify that <code>prevent_audit_log_modification</code> trigger is active. | No fix needed—this confirms tamper-proof immutability is functioning! |

